Great Place To Work. Certified FEB 2023 – FEB 2024 INDIA

UPES

UNIVERSITY OF TOMORROW

nirf RANKING 2023

#RANKED 52 IN INDIA

#University Category

QS WORLD UNIVERSITY RANKINGS

NO.1 PVT. UNIVERSITY IN ACADEMIC REPUTATION IN INDIA

NAAC

ACCREDITED GRADE 'A' BY NAAC

PERFECT SCORE OF 150/150 AS A TESTAMENT TO EXECEPTIONAL E-LEARNING METHODS

UPES UNIVERSITY OF TOMORROW

Elements of AIML

Unit 03 – Lecture 02: Train/Test/Validation Split & Cross-Validation

Tofik Ali

School of Computer Science, UPES Dehradun

March 28, 2026

Topic focus: **evaluation without fooling yourself** (splits, CV, leakage)

Repository: `https://github.com/tali7c/Elements-of-AIML`

Sources: Mueller and Massaron, *Machine Learning for Dummies*, For Dummies, 2016.;Mitchell, *Machine Learning*, selected chapters on learning basics and evaluation.;Scikit-learn documentation, accessed 2026-02-28.

Quick Links

Splits Cross-Validation Leakage Code Summary

Agenda

Splits

Cross-validation

Leakage

Code

Summary

Learning Outcomes

By the end of this lecture, you should be able to:

- Explain why we split data into train/validation/test sets.

Learning Outcomes

By the end of this lecture, you should be able to:

- Explain why we split data into train/validation/test sets.
- Describe what decisions are allowed using each split.

Learning Outcomes

By the end of this lecture, you should be able to:

- Explain why we split data into train/validation/test sets.
- Describe what decisions are allowed using each split.
- Explain k-fold cross-validation and stratification.

Learning Outcomes

By the end of this lecture, you should be able to:

- Explain why we split data into train/validation/test sets.
- Describe what decisions are allowed using each split.
- Explain k-fold cross-validation and stratification.
- Identify common data leakage patterns and avoid them.

Before We Split: What Must Already Be Clear?

- From Lecture 01: know what the **features**, **labels**, and **task type** are.

Before We Split: What Must Already Be Clear?

- From Lecture 01: know what the **features**, **labels**, and **task type** are.
- Choose the split strategy *after* identifying the task: classification, regression, grouped data, or time series.

Before We Split: What Must Already Be Clear?

- From Lecture 01: know what the **features**, **labels**, and **task type** are.
- Choose the split strategy *after* identifying the task: classification, regression, grouped data, or time series.
- Honest evaluation means the **test set stays untouched** until the end.

Abbreviations & Symbols

- Train/Val/Test: training/validation/testing splits

Abbreviations & Symbols

- Train/Val/Test: training/validation/testing splits
- CV: cross-validation

k: number of folds

Abbreviations & Symbols

- Train/Val/Test: training/validation/testing splits
- CV: cross-validation
- $\hat{y}$: predictions

k: number of folds

metric: accuracy/F1/RMSE, etc.

Abbreviations & Symbols

- Train/Val/Test: training/validation/testing splits
- CV: cross-validation k: number of folds
- $\hat{y}$: predictions metric: accuracy/F1/RMSE, etc.
- Hyperparameter: setting chosen by us (e.g., depth, C, λ)

Abbreviations & Symbols

- Train/Val/Test: training/validation/testing splits
- CV: cross-validation k: number of folds
- $\hat{y}$: predictions metric: accuracy/F1/RMSE, etc.
- Hyperparameter: setting chosen by us (e.g., depth, C, λ)
- Leakage: using information unavailable at prediction time

Why Split the Dataset?

- Goal: estimate performance on **unseen** data (generalization).

Why Split the Dataset?

- Goal: estimate performance on **unseen** data (generalization).
- If we evaluate on training data, we can be fooled by **overfitting**.

Why Split the Dataset?

- Goal: estimate performance on **unseen** data (generalization).
- If we evaluate on training data, we can be fooled by **overfitting**.
- Splits create a clean separation between learning and evaluation.

Train vs Validation vs Test

- **Train set**: fit model parameters.

Rule: do not repeatedly tune using the test set (it becomes a validation set).

Train vs Validation vs Test

- **Train set**: fit model parameters.
- **Validation set**: choose model/hyperparameters (tuning).

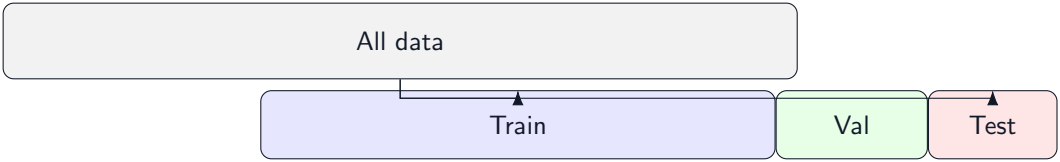
Rule: do not repeatedly tune using the test set (it becomes a validation set).

Train vs Validation vs Test

- **Train set**: fit model parameters.
- **Validation set**: choose model/hyperparameters (tuning).
- **Test set**: final unbiased evaluation (use once at the end).

Rule: do not repeatedly tune using the test set (it becomes a validation set).

Split Diagram (Hold-out Evaluation)



Tune on **Val** (or CV), report final score on **Test** once.

Cross-Validation (k-fold)

- Split training data into k folds.

Benefits: more stable estimate, better use of limited data.

Cross-Validation (k-fold)

- Split training data into k folds.
- Train on $k-1$ folds, validate on the remaining fold.

Benefits: more stable estimate, better use of limited data.

Cross-Validation (k-fold)

- Split training data into k folds.
- Train on $k-1$ folds, validate on the remaining fold.
- Repeat for all folds; average performance.

Benefits: more stable estimate, better use of limited data.

k-Fold Intuition (What happens each round?)

Round 1: [Val] — Train — Train — Train — Train
Round 2: Train — [Val] — Train — Train — Train
Round 3: Train — Train — [Val] — Train — Train
...

Average the validation metric over rounds; then retrain on all training data with chosen settings.

Worked Story: Churn Prediction

- Suppose 1,000 customers are available and only about 12% churn.

Worked Story: Churn Prediction

- Suppose 1,000 customers are available and only about 12% churn.
- Split into train/test first, then use **stratified 5-fold CV** inside training.

Worked Story: Churn Prediction

- Suppose 1,000 customers are available and only about 12% churn.
- Split into train/test first, then use **stratified 5-fold CV** inside training.
- Example validation F1 scores: 0.71, 0.75, 0.73, 0.70, 0.76.

Worked Story: Churn Prediction

- Suppose 1,000 customers are available and only about 12% churn.
- Split into train/test first, then use **stratified 5-fold CV** inside training.
- Example validation F1 scores: 0.71, 0.75, 0.73, 0.70, 0.76.
- Choose the model using the average CV score; report the test score only once at the end.

Stratification (Classification)

For classification, ensure each fold keeps class proportions similar to the full dataset.

Why it matters

If a fold has almost no positive examples, metrics like recall become meaningless.

Data Leakage: A Silent Killer

Leakage = using information that would not be available at prediction time.

- Using post-event fields (e.g., "approved_date" to predict approval)

Data Leakage: A Silent Killer

Leakage = using information that would not be available at prediction time.

- Using post-event fields (e.g., "approved_date" to predict approval)
- Preprocessing using the full dataset before splitting (e.g., scaling using all data)

Data Leakage: A Silent Killer

Leakage = using information that would not be available at prediction time.

- Using post-event fields (e.g., "approved_date" to predict approval)
- Preprocessing using the full dataset before splitting (e.g., scaling using all data)
- Duplicates across train and test (near-identical samples)

Leakage Prevention Rules (Checklist)

- Split first, then fit preprocessing on **train only**

Leakage Prevention Rules (Checklist)

- Split first, then fit preprocessing on **train only**
- Do feature selection inside CV (not before)

Leakage Prevention Rules (Checklist)

- Split first, then fit preprocessing on **train only**
- Do feature selection inside CV (not before)
- Keep duplicates/groups in a single split (GroupKFold when needed)

Leakage Prevention Rules (Checklist)

- Split first, then fit preprocessing on **train only**
- Do feature selection inside CV (not before)
- Keep duplicates/groups in a single split (GroupKFold when needed)
- For time series, split by time (no future $\rightarrow$ past)

Minimal Code Pattern (scikit-learn)

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Use `stratify=y` for classification tasks. For regression, omit it and choose a split strategy that matches the data source.

Leakage-Safe Pattern (Pipeline)

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression())
])
```

With CV, the scaler is fit **inside each fold** (safer).

k-Fold Cross-Validation (sketch)

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for train_idx, val_idx in cv.split(X, y):
    X_tr, X_val = X[train_idx], X[val_idx]
    y_tr, y_val = y[train_idx], y[val_idx]
```

Summary

- Splits protect us from overfitting and give honest evaluation.
- Validation is for tuning; test is for the final report.
- Cross-validation improves stability when data is limited.
- Leakage can make results look great but fail in production.

Sources: Mueller and Massaron, *Machine Learning for Dummies*, For Dummies, 2016.;Mitchell, *Machine Learning*, selected chapters on learning basics and evaluation.;Scikit-learn documentation, accessed 2026-02-28.

Exit Questions

1. Why should the test set be used only once at the end?
2. What is cross-validation and when do we use it?
3. Give two examples of data leakage.
4. What does stratification mean and why is it used?